W6 — React Native Temel Bileşenler | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W6
Kapsam: `View`, `Text`, `Image`, `TextInput`, `Button`, `ScrollView`, `FlatList`, `TouchableOpacity`, `SafeAreaView`, `Picker`, `RadioButton`, `Checkbox`.
Hedef: Hangi bileşen ne işe yarar, hangi prop ile kullanılır, sık yapılan hatalar ve **Expo Snack** `App.js` örnekleri.

> **Kaynak notu:** `Picker` ve `Checkbox` için ayrı paket gerekir. `RadioButton` çekirdek React Native'de hazır bir bileşen değildir; bu rehberde `Pressable` ile yapılır. `SafeAreaView` için modern projelerde `react-native-safe-area-context` tercih edilir. Bu farklar ders deposundaki W6 anlatımlarıyla uyumludur.

1. Bileşenleri 30 saniyede tanı

Bileşen	Görevi	En önemli prop/olay
`View`	Kutu, satır, kolon, düzen	`style`
`Text`	Yazı gösterme	`style`, `numberOfLines`
`Button`	Hızlı hazır buton	`title`, `onPress`, `disabled`
`TouchableOpacity`	Özelleştirilebilir basılabilir alan	`onPress`, `activeOpacity`
`Pressable`	Esnek basılma davranışı	`onPress`, `onPressIn`, `style`
`Image`	Yerel veya internet görseli	`source`, `style`, `resizeMode`
`TextInput`	Kullanıcıdan metin alma	`value`, `onChangeText`, `placeholder`
`ScrollView`	İçeriği kaydırma	`horizontal`, `contentContainerStyle`
`FlatList`	Çok öğeli, performanslı liste	`data`, `renderItem`, `keyExtractor`
`SafeAreaView`	Çentik/status bar boşluğu	`edges`, `style` (safe-area-context)
`Picker`	Bir seçenek seçme	`selectedValue`, `onValueChange`
`RadioButton`	Tek seçenekli özel kontrol	`Pressable` ile seçili state
`Checkbox`	Aç/kapat, birden çok işaret	`value`, `onValueChange`

2. `View` — yerleşim kutusu

`View` bileşenleri gruplayan ana kapsayıcıdır. `div` karşılığına yakındır ama React Native **Flexbox** ile yerleşim yapar.

```
<View style={{ flex: 1, padding: 20, backgroundColor: '#F8FAFC' }}>
  <Text>İçerik</Text>
</View>
```

`flexDirection: 'row'` yatay, `'column'` dikey yerleşim. `justifyContent` ana eksen, `alignItems` çapraz eksen hizalamasıdır. `View` içine doğrudan `Merhaba` yazısı koyma; `<Text>Merhaba</Text>` kullan.

3. `Text` — metin, satır kısıtlama

```
<Text style={{ fontSize: 24, fontWeight: 'bold', color: '#1D4ED8' }}>
  Hoş geldin ÜNAL
</Text>
<Text numberOfLines={1} ellipsizeMode="tail">
  Çok uzun bir açıklama gerektiğinde sonunu kısaltabiliriz.
</Text>
```

`numberOfLines={1}` tek satıra sınırlar. `ellipsizeMode="tail"` gerekirse sonunu `...` ile kısaltır. İç içe `Text` ile farklı renk/kalınlıkta metin hazırlanabilir.

4. `Button`, `TouchableOpacity`, `Pressable` — tıklamalar

Hazır `Button`: Kullanması en kolay, stil seçenekleri kısıtlıdır.

```
<Button title="Göster" onPress={() => Alert.alert('Merhaba!')} />
<Button title="Pasif" onPress={() => {}} disabled />
```

Özel buton `TouchableOpacity`: İçine `Text`, `Image`, `View` koyabilirsin.

```
<TouchableOpacity
  activeOpacity={0.7}
  onPress={() => Alert.alert('Butona basıldı')}
  style={{ backgroundColor: '#2563EB', padding: 16, borderRadius: 12 }}
>
  <Text style={{ color: 'white', textAlign: 'center' }}>Özel Buton</Text>
</TouchableOpacity>
```

`Pressable`: Yeni etkileşimlerde tercih edilebilir. `pressed` durumuna göre stil değiştirebilir.

```
<Pressable
  onPress={() => console.log('Basılma tamamlandı')}
  style={({ pressed }) => ({
    padding: 14,
    borderRadius: 10,
    backgroundColor: pressed ? '#1D4ED8' : '#60A5FA',
  })}
>
  <Text style={{ color: '#fff' }}>Bana bas</Text>
</Pressable>
```

Tuzak: `onPress={() => islem()}` doğrudur; `onPress={islem()}` fonksiyon sonucu verir ve erken çalıştırabilir.

5. `TextInput` — kontrollü veri girişi

```
const [ad, setAd] = useState('');
<TextInput
  value={ad}
  onChangeText={setAd}
  placeholder="Adını yaz"
  autoCapitalize="words"
  style={{ borderWidth: 1, borderColor: '#CBD5E1', padding: 12 }}
/>
<Text>Girilen: {ad}</Text>
```

Prop	İşlevi
`placeholder`	Boşken görünen ipucu
`value`	State'te tutulan değer
`onChangeText`	Metin her değiştiğinde günceller
`keyboardType="email-address"`	E-posta klavyesi
`keyboardType="numeric"`	Sayısal klavye (değer yine string)
`secureTextEntry`	Şifreyi gizler
`multiline`	Çok satırlı giriş
`maxLength={20}`	En fazla 20 karakter

Önemli: `TextInput` için web'deki `onChange={e => setAd(e.target.value)}` kalıbı yerine genellikle **`onChangeText={setAd}`** kullanılır.

6. `Image` — resim ekleme

```
<Image
  source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
  style={{ width: 120, height: 120, borderRadius: 12 }}
  resizeMode="contain"
/>
```

Yerel resim: `source={require('./assets/logo.png')}` (resmin dosyada mevcut olması şart). İnternet görseli için `source={{ uri: 'https://...' }}`. `resizeMode`: `cover` alanı doldurup kırpabilir, `contain` tamamını gösterir. Özellikle uzak resimde `width` ve `height` belirlemek gerekir.

7. `ScrollView` — kısa, kaydırılabilir içerik

```
<ScrollView contentContainerStyle={{ padding: 20, gap: 10 }}>
  <Text>Üst yazı</Text>
  <Text>Çok sayıda içerik...</Text>
</ScrollView>
```

Kısa formlar ve bilgi sayfaları için uygundur. İçindeki **tüm** çocukları render eder; binlerce kayıt varsa performans için `FlatList` kullan. `horizontal` yatay kaydırma sağlar.

8. `FlatList` — dizi listeleme

```
const urunler = [
  { id: 'a', ad: 'Kalem' },
  { id: 'b', ad: 'Defter' },
];
<FlatList
  data={urunler}
  keyExtractor={item => item.id}
  renderItem={({ item }) => (
    <Text style={{ padding: 12 }}>{item.ad}</Text>
  )}
/>
```

Prop açıklaması: `data` listedeki dizi; `renderItem` her öğenin görünümü; `keyExtractor` benzersiz kimlik; `ListEmptyComponent` liste boşken gösterim; `ItemSeparatorComponent` ayırıcı; `horizontal` yatay liste; `onEndReached` sonlara yaklaşıldığında olay; `refreshing` / `onRefresh` yenileme davranışı.

Sınavda kritik ayrım: `ScrollView` tüm çocukları birden oluşturur. `FlatList` sanallaştırma (virtualization) ile uzun listelerde daha verimlidir. `FlatList` bir öğeye otomatik `key` bulmaya çalışabilir; açıkça `keyExtractor` vermek daha sağlamdır.

9. `SafeAreaView` — iPhone çentiği ve güvenli alan

Modern kullanım için Snack Dependencies içine **`react-native-safe-area-context`** ekle (Snack'te kurulu olabilir):

```
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1 }}>
        <Text>Çentikten ve alt göstergeden uzak içerik</Text>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

Not: `SafeAreaView` ismi geçmişte `react-native` içinde de yer alır; güncel projelerde `react-native-safe-area-context` çözümü tercih edilir. `SafeAreaProvider` çoğu tam projede kök uygulama düzeyinde bir kez tanımlanır.

10. `Picker` — seçmeli menü (ek paket)

Snack **Dependencies** içine `@react-native-picker/picker` ekle:

```
import React, { useState } from 'react';
import { View, Text } from 'react-native';
import { Picker } from '@react-native-picker/picker';

export default function App() {
  const [sehir, setSehir] = useState('istanbul');
  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 20 }}>
      <Text>Seçilen şehir: {sehir}</Text>
      <Picker selectedValue={sehir} onValueChange={setSehir}>
        <Picker.Item label="İstanbul" value="istanbul" />
        <Picker.Item label="Ankara" value="ankara" />
      </Picker>
    </View>
  );
}
```

`selectedValue` kontrollü seçili değerdir; `onValueChange` seçim değişince state'i günceller. Yerleşik React Native çekirdeğinde `Picker` artık bulunmaz; doğru paketten import et.

11. `RadioButton` — tek seçenek (Pressable ile)

Çekirdek RN'de hazır `RadioButton` bulunmadığından, tek seçim için state + `Pressable` kullanabiliriz:

```
const [secim, setSecim] = useState('kolay');
{['kolay', 'orta', 'zor'].map(seviye => (
  <Pressable key={seviye} onPress={() => setSecim(seviye)}
    style={{ padding: 12, flexDirection: 'row', alignItems: 'center' }}>
    <Text>{secim === seviye ? '🔘' : '⚪'} {seviye}</Text>
  </Pressable>
))}
```

Tek bir `secim` değeri olduğu için aynı anda yalnızca bir seçenek aktif olabilir. Daha kapsamlı ve erişilebilir radyo kontrolleri için desteklenen UI kütüphanesi de kullanılabilir.

12. `Checkbox` — çoklu seçim veya onay (ek paket)

Snack **Dependencies** içine `expo-checkbox` ekle:

```
import React, { useState } from 'react';
import { View, Text } from 'react-native';
import Checkbox from 'expo-checkbox';

export default function App() {
  const [onay, setOnay] = useState(false);
  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 30, gap: 15 }}>
      <Checkbox value={onay} onValueChange={setOnay} color={onay ? '#2563EB' : undefined} />
      <Text>{onay ? 'Onaylandı' : 'Henüz onaylanmadı'}</Text>
    </View>
  );
}
```

`value`: true/false; `onValueChange`: kullanıcı değiştirdiğinde çalışır. `RadioButton` **tek seçenek**; checkbox **birden çok bağımsız seçenek** için uygundur.

13. TAM UYGULAMA 1 — Arama + liste + silme (`App.js`)

Bu örnek **yalnızca React ve react-native** kullanır. Expo Snack'e doğrudan yapıştır. `TextInput`, `Button`, `FlatList`, `Pressable`, `useState` birleşir.

```
import React, { useState } from 'react';
import {
  View, Text, TextInput, Button, FlatList, Pressable, StyleSheet,
} from 'react-native';

const ilkKisiler = [
  { id: '1', ad: 'Ünal' },
  { id: '2', ad: 'Ayşe' },
  { id: '3', ad: 'Can' },
];

export default function App() {
  const [kisiler, setKisiler] = useState(ilkKisiler);
  const [arama, setArama] = useState('');
  const [yeniAd, setYeniAd] = useState('');

  const gorunen = kisiler.filter(k =>
    k.ad.toLocaleLowerCase('tr-TR').includes(arama.toLocaleLowerCase('tr-TR'))
  );

  function ekle() {
    if (!yeniAd.trim()) return;
    setKisiler(eski => [...eski, { id: String(Date.now()), ad: yeniAd.trim() }]);
    setYeniAd('');
  }
  function sil(id) {
    setKisiler(eski => eski.filter(k => k.id !== id));
  }

  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Kişi Listesi</Text>
      <TextInput style={styles.girdi} placeholder="Kişi ara..."
        value={arama} onChangeText={setArama} />
      <TextInput style={styles.girdi} placeholder="Yeni kişi adı"
        value={yeniAd} onChangeText={setYeniAd} />
      <Button title="Kişi ekle" onPress={ekle} />
      <FlatList
        data={gorunen}
        keyExtractor={item => item.id}
        ListEmptyComponent={<Text>Kayıt bulunamadı</Text>}
        renderItem={({ item }) => (
          <View style={styles.satir}>
            <Text style={styles.ad}>{item.ad}</Text>
            <Pressable onPress={() => sil(item.id)}>
              <Text style={styles.sil}>SİL</Text>
            </Pressable>
          </View>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, padding: 24, paddingTop: 65, gap: 12 },
  baslik: { fontSize: 27, fontWeight: 'bold', marginBottom: 6 },
```

```
  girdi: { borderWidth: 1, borderColor: '#94A3B8', padding: 12, borderRadius: 10 },
  satir: { padding: 14, flexDirection: 'row', justifyContent: 'space-between',
    alignItems: 'center', backgroundColor: '#EFF6FF', marginTop: 8, borderRadius: 10 },
  ad: { fontSize: 17 },
  sil: { color: '#DC2626', fontWeight: 'bold' },
});
```

Neyi öğrenirsin? Ekran başlığı, kontrollü input, anlık filtreleme, yeni öğe ekleme, `keyExtractor`, öğe silme. `Date.now()` sınıf içi küçük örneklerde kimlik için işe yarar; gerçek kalıcı veride benzersiz ID stratejisi kullanılmalıdır.

14. TAM UYGULAMA 2 — Buton + görsel + Switch benzeri kontrol

Bu örnek ekstra paket olmadan çalışır. Özellikle `TouchableOpacity`, `Image`, `Text` ve koşullu gösterimi birlikte çalıştırır.

```
import React, { useState } from 'react';
import { View, Text, Image, TouchableOpacity, StyleSheet } from 'react-native';

export default function App() {
  const [begeni, setBegeni] = useState(false);
  const [puan, setPuan] = useState(0);
  return (
    <View style={styles.ekran}>
      <Image source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
        style={{ width: 100, height: 100 }} resizeMode="contain" />
      <Text style={styles.baslik}>React Native Kartı</Text>
      <Text>Puan: {puan}</Text>
      <TouchableOpacity style={styles.buton} onPress={() => setPuan(x => x + 1)}>
        <Text style={styles.butonYazi}>+1 PUAN</Text>
      </TouchableOpacity>
      <TouchableOpacity onPress={() => setBegeni(x => !x)}>
        <Text style={{ fontSize: 18 }}>{begeni ? '❤️ Beğenildi' : '🤍 Beğen'}</Text>
      </TouchableOpacity>
    </View>
  );
}
const styles = StyleSheet.create({
  ekran: { flex: 1, alignItems: 'center', justifyContent: 'center', gap: 18 },
  baslik: { fontSize: 24, fontWeight: 'bold' },
  buton: { backgroundColor: '#2563EB', padding: 16, borderRadius: 12 },
  butonYazi: { color: '#fff', fontWeight: 'bold' },
});
```

15. Bileşen seçme kararı

İhtiyaç	En uygun bileşen
Bir başlık göstermek	`Text`
İçeriği iki sütuna bölmek	`View` + Flexbox
Tek seferlik tıklama	`Button`

İhtiyaç	En uygun bileşen
Özel renk/boyutlu buton	`Pressable / TouchableOpacity`
Ad, e-posta, şifre almak	`TextInput`
Kullanıcı resmini göstermek	`Image`
6 kartı kaydırmak	`ScrollView`
500 kişi listelemek	`FlatList`
iPhone çentiğine denk gelmemek	Safe area context `SafeAreaView`
Şehir seçmek	`Picker`
Seviye seçmek (bir tane)	RadioButton yaklaşımı
Sözleşmeyi onaylatmak	`Checkbox`

16. Sık hatalar / sınav tuzakları

H1. `FlatList`'e `data` verip `renderItem` vermemek → öğeler çizilmez.
H2. Uzun listeyi `ScrollView` ile şişirmek → gereksiz render, düşük performans.
H3. `Image` için `width/height` belirtmemek → uzak resim görünmeyebilir.
H4. `TextInput` değerini `onChangeText` ile state'e yazmamak → kontrollü giriş çalışmaz.
H5. `Button` içine `<Text>` çocuk koymak → RN `Button` `title` prop'u kullanır.
H6. `Picker` veya `Checkbox`'ı `react-native`'den import etmek → doğru dış paketi kullan.
H7. RadioButton'ı standart `react-native` export'u sanmak → özel Pressable/UI kütüphanesi gerekir.
H8. `SafeAreaView` sürümüne bakmamak → modern safe-area-context paketi önerilir.

17. Kendini test et (cevap anahtarlı)

S1. Çok sayıda kişiyi performanslı listelemek için? **C:** `FlatList`.

S2. Kullanıcının yazdığı metni hangi prop ile alırsın? **C:** `onChangeText`.

S3. Buton basıldığında çalışan prop? **C:** `onPress`.

S4. Bir resmin tamamı kırpılmadan gözüksün istiyorsan? **C:** `resizeMode="contain"`.

S5. Bir tane seçenek mi, çoklu seçenek mi? **C:** Tek: RadioButton; çoklu/bağımsız: Checkbox.

S6. `keyExtractor` neden? **C:** Liste öğelerini tutarlı biçimde kimliklendirmek.

S7. `View`'de iki kutuyu yan yana nasıl koyarsın? **C:** `flexDirection: 'row'`.

S8. `Button` rengini ve içine ikonlu özel tasarımı rahatça yapmak için? **C:** `Pressable` / `TouchableOpacity` + `Text` / `Image`.

S9. Expo Snack'te dış paket gerekiyorsa nereden eklenir? **C:** `Dependencies` bölümünden.

S10. `SafeAreaProvider` niçin? **C:** Güvenli alan bilgisi için kapsayıcı sağlar.

18. Son dakika kopyala-bak tablosu

Kod	Amacı
`<Text>Yazı</Text>`	Ekrana yazı
`<View style={{flex:1}}>`	Ana kapsayıcı
`<Button title="Bas" onPress={islem} />`	Hazır buton
`<TextInput value={x} onChangeText={setX} />`	Metin girişi
`<Image source={{uri:url}} style={{width:90,height:90}} />`	İnternet resmi
`<ScrollView>...</ScrollView>`	Kaydırma
`<FlatList data={a} renderItem={({item}) => ...} keyExtractor={x => x.id} />`	Liste
`{aktif && <Text>Görünür</Text>}`	Koşullu görünüm
`setListe(l => l.filter(x => x.id !== id))`	Öğeyi silme
`setListe(l => [...l, eleman])`	Öğeyi ekleme

Kaynak dosyalar: W6/README.md, W6/components klasörü.
Bu dosya kaynak ders konularına göre hazırlanmış Türkçe açıklamalı çalışmadır. Kodları Expo Snack'e kopyalarken "tam App.js" örneklerini birbirinden ayrı dene.